



UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE CIENCIAS
DEPARTAMENTO DE MATEMÁTICAS

**REPORTE TÉCNICO: SIMULADOR
INTERACTIVO DE CONFLICTO
ARANCELARIO**

Estudiante:

Guerrero Rodriguez, Emanuel David

Documento: 1006821727

Bogotá D.C., Colombia
Julio de 2026

SIMULADOR INTERACTIVO DE CONFLICTO ARANCELARIO

Reporte técnico y académico correspondiente al desarrollo y programación de un modelo de simulación continua basado en optimización matemática y cálculo diferencial, aplicado a la evaluación del impacto arancelario externo y la eficiencia del gasto fiscal.

Autor:

Guerrero Rodriguez, Emanuel David

Docente:

Mina Ladino, Jhon Cristian

**UNIVERSIDAD NACIONAL DE COLOMBIA
FACULTAD DE CIENCIAS**

Bogotá D.C., Colombia

2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Marco teórico	3
4. Fundamentación matemática y desarrollo del modelo	3
4.1. Funciones de producción y sensibilidad	3
4.2. El factor de retaliación	4
4.3. Costo fiscal y arancel efectivo	4
4.4. Optimización de bienestar social	5
5. Diseño de software y programación del código	5
5.1. Justificación de decisiones de diseño del modelo	5
5.2. Herramientas usadas	5
5.3. Explicación de los algoritmos de optimización	6
5.4. Explicación de módulos del código	6
5.4.1. Módulo 1: Importación de dependencias y configuración del entorno	6
5.4.2. Módulo 2: Declaración de variables simbólicas y funciones base . . .	7
5.4.3. Módulo 3: Construcción de la interfaz gráfica interactiva o GUI . .	7
5.4.4. Módulo 4: El algoritmo de cálculo híbrido Simbólico-Vectorizado . .	7
5.4.5. Módulo 5: Renderizado del diagnóstico macroeconómico	8
5.4.6. Módulo 6: Gestor de eventos asíncronos y calibración en tiempo real	8
5.4.7. Módulo 7: Controlador paramétrico de escenarios económicos	9
5.4.8. Módulo 8: Motor de compresión y exportación documental	9
5.5. Modos de simulación y escenarios	9
5.6. Resultados analíticos del modelo	10
6. Conclusiones	11
7. Ficha técnica del software	12
Referencias	13

1. Introducción

En un entorno económico globalizado, las guerras arancelarias y las políticas proteccionistas representan amenazas significativas para el bienestar social de las naciones en vías de desarrollo. Colombia no es ajena a esta realidad; el país cuenta con un historial de tensiones y barreras comerciales impuestas por socios estratégicos, con episodios y fricciones recientes que involucran a naciones como Estados Unidos y Ecuador. A partir de estos eventos, que no solo destruyen la demanda externa de sectores primarios sino que también fuerzan al Estado a intervenir mediante subsidios compensatorios, surge una necesidad primaria en la agenda pública y educativa por entender las dinámicas de este fenómeno.

Este documento describe la arquitectura matemática y la funcionalidad del Simulador Interactivo de Conflicto Arancelario, una herramienta computacional sencilla y práctica, diseñada principalmente para su uso en entornos académicos. Su propósito es evaluar de manera cuantitativa el impacto de ataques arancelarios externos, el daño autoinfligido por la retaliación comercial, y la eficiencia del gasto público destinado a mitigar estas pérdidas.

Para lograr este análisis, resulta estrictamente necesario hacer uso del cálculo diferencial. Mediante esta herramienta analítica, el simulador permite identificar el punto exacto de equilibrio donde el costo marginal de la intervención estatal se iguala con el beneficio social recuperado. Si bien la aplicación de este modelo continuo brinda resultados que pueden ser limitados en comparación con simulaciones macroeconómicas más complejas, la herramienta cumple a la perfección con su objetivo principal: servir como un abrebocas pedagógico e intuitivo para acercar a poblaciones poco informadas o no familiarizadas con la temática a la toma de decisiones basada en la evidencia matemática.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar un entorno interactivo de simulación computacional que permita modelar el impacto macroeconómico de las fricciones comerciales y optimizar la asignación presupuestaria de subsidios estatales mediante cálculo diferencial.

2.2. Objetivos específicos

- Estructurar un modelo matemático continuo, fundamentado en el cálculo diferencial, que traduzca las complejas dinámicas de fricción arancelaria y retaliación comercial en un sistema simplificado y fácil de asimilar para contextos académicos.
- Implementar un motor simbólico de optimización en Python que resuelva analíticamente el punto exacto de equilibrio entre el costo marginal fiscal y el beneficio social, evidenciando la utilidad práctica de las derivadas.
- Desarrollar una interfaz gráfica interactiva e intuitiva que funcione como puente pedagógico, permitiendo a poblaciones no especializadas visualizar y comprender escenarios de conflicto económico de forma accesible.

3. Marco teórico

El análisis de las políticas proteccionistas exige equilibrar la teoría macroeconómica con la precisión matemática. En una guerra arancelaria, los impuestos extranjeros encarecen las exportaciones y contraen fuertemente la demanda internacional. Para mitigar este daño, el Estado suele intervenir con subsidios compensatorios que protegen la industria local, pero que a su vez generan un alto costo fiscal. Encontrar el punto de equilibrio exacto entre el rescate del sector productivo y la sostenibilidad de las arcas públicas hace indispensable el uso de modelos cuantitativos dinámicos.

Para lograr esta precisión, el cálculo diferencial resulta una herramienta fundamental. En lugar de asignar recursos públicos mediante estimaciones empíricas o ensayo y error, el uso de derivadas permite medir la velocidad exacta a la que una industria pierde competitividad ante los aumentos arancelarios. Asimismo, el problema de maximizar el bienestar nacional se resuelve de forma analítica al encontrar el punto donde la tasa de cambio de la función se anula. En este óptimo matemático, el costo marginal de otorgar un subsidio adicional se iguala perfectamente con el beneficio marginal de las exportaciones rescatadas.

En términos puramente económicos, este andamiaje matemático permite cuantificar la vulnerabilidad de cada mercado y los daños colaterales del conflicto. Por un lado, evidencia cómo sectores primarios (como el agropecuario) presentan una alta elasticidad y caen rápidamente por la abundancia de sustitutos globales, mientras que la industria tecnológica muestra mayor resiliencia. Por otro lado, el modelo incorpora el “efecto búmeran” de la retaliación comercial: demuestra que cuando un país decide contraatacar con aranceles propios, inevitablemente genera fricciones logísticas e inflación interna que terminan afectando a su propia economía.

4. Fundamentación matemática y desarrollo del modelo

El sistema se basa en un modelo de optimización continua compuesto por tres subsistemas principales: la oferta productiva (sector agropecuario y tecnología), la fricción internacional, y la caja fiscal. A continuación, se desglosa la formulación analítica de cada componente matemático —desde las curvas de sensibilidad sectorial hasta la función de bienestar social— exponiendo cómo el cálculo diferencial permite deducir rigurosamente las políticas fiscales óptimas de intervención.

4.1. Funciones de producción y sensibilidad

La cantidad de unidades exportables dependientes del arancel extranjero evaluado (t) se define empíricamente. El usuario puede modificar las constantes, pero las estructuras canónicas propuestas son:

- **Sector Agropecuario:**

$$Q_{agro}(t) = a - b \cdot F \cdot t^2 \tag{1}$$

La elección de una estructura cuadrática para el sector agropecuario responde a la hipótesis de **rendimientos marginales decrecientes y alta sustituibilidad de bienes primarios**. Como describe Krugman [1, p. 490], la imposición de barreras comerciales en el transporte de mercancías primarias y agrícolas introduce un colapso abrupto o “*cliff*” (acantilado) en el mercado. En el simulador, aproximamos analíticamente este fenómeno de desplome acelerado y no lineal a través de este término de segundo orden. Matemáticamente, la sensibilidad (la tasa de cambio del volumen exportado ante variaciones del arancel) se obtiene mediante la primera derivada:

$$\frac{dQ_{agro}}{dt} = -2 \cdot b \cdot F \cdot t \quad (2)$$

Esto demuestra que el impacto marginal del arancel no es constante, sino que se acelera a medida que el arancel aumenta ($\frac{d^2Q_{agro}}{dt^2} = -2bF < 0$).

■ Sector Tecnológico:

$$Q_{tech}(t) = c - d \cdot F \cdot t \quad (3)$$

Por el contrario, el sector de base tecnológica se modela bajo una estructura lineal con una tasa de pérdida marginal constante:

$$\frac{dQ_{tech}}{dt} = -d \cdot F \quad (4)$$

Económicamente, esto representa un sector con **baja elasticidad-precio de la demanda debido a la diferenciación de producto y altas barreras de entrada**. Siguiendo la modelación fundacional de Melitz [2, p. 1700] sobre industrias con bienes diferenciados, las firmas de alta tecnología poseen la capacidad de absorber de forma estable parte de las variaciones del coste arancelario externo mediante sus márgenes comerciales, permitiendo que el volumen caiga de forma lineal en lugar de colapsar abruptamente.

4.2. El factor de retaliación

La decisión soberana de responder a un ataque extranjero con un arancel de represalia (R) genera fricción logística, diplomática y aduanera que se modela formalmente mediante el Factor de Guerra ($F(R)$):

$$F(R) = 1 + (R \times 0,005) \quad (5)$$

Esta estructura funciona modelando matemáticamente el concepto de “costos de iceberg” introducido por Samuelson [3, p. 268], donde las fricciones recíprocas actúan derribando una fracción del valor del comercio, de manera que a mayor agresividad arancelaria recíproca (R), mayor es la distorsión operativa interna ($\frac{dF}{dR} = 0,005 > 0$).

4.3. Costo fiscal y arancel efectivo

El Estado puede inyectar un porcentaje de subsidio (s) para reducir artificialmente el arancel extranjero que perciben los productores locales. El arancel efectivo resultante es $t_{eff} = \max(0, t - s)$. El costo de inyectar este subsidio sigue una trayectoria cuadrática:

$$C(s) = k \cdot s^2 \quad (6)$$

Este comportamiento obedece a la teoría empírica de la pérdida irrecuperable de eficiencia, o el “Triángulo de Harberger” [4, p. 59], la cual demuestra rigurosamente que el costo social y el desperdicio burocrático de una intervención fiscal crecen de manera proporcional al cuadrado de la tasa del subsidio inyectado.

4.4. Optimización de bienestar social

El bienestar social (B) se define como el valor de las exportaciones del sector vulnerable (Agropecuaria) por un precio constante (P), menos el costo fiscal, sumado al recaudo por retaliación:

$$B(s) = P \cdot Q_{agro}(t_{eff}) - C(s) + (R \times 150) \quad (7)$$

Para hallar el subsidio óptimo, el motor de cálculo obtiene las derivadas analíticas $\frac{dQ_{agro}}{dt}$ y $\frac{dC}{ds}$, y busca el punto donde la derivada se anula ($\frac{dB}{ds} = 0$). Este abordaje numérico está directamente fundamentado en los modelos de política comercial estratégica desarrollados por Brander y Spencer [5, p. 88], quienes resuelven analíticamente el nivel de subsidio óptimo igualando el beneficio marginal de salvar exportaciones al costo marginal fiscal:

$$\frac{dB}{ds} = P \cdot \left(-\frac{dQ_{agro}}{dt} \right) - \frac{dC}{ds} = 0 \quad (8)$$

5. Diseño de software y programación del código

Para comprender el comportamiento dinámico e integrado de estas variables, esta sección aborda las decisiones de diseño arquitectónico y las estrategias algorítmicas que dotan de robustez, estabilidad y usabilidad al simulador en un entorno interactivo.

5.1. Justificación de decisiones de diseño del modelo

El modelo fue diseñado permitiendo la edición libre de las funciones matemáticas en tiempo real a través de la interfaz. Para mitigar errores de digitación por parte de los usuarios, se incorporó un teclado matemático virtual y un sistema de validación sintáctica.

Adicionalmente, se incluyó una restricción explícita de caja: el límite de presupuesto. Si el costo de alcanzar el bienestar óptimo teórico excede el presupuesto del Estado, el simulador trunca la solución asumiendo que el Estado gasta hasta agotar sus reservas, graficando una “Zona Fiscal Restringida”. A nivel de seguridad y exportación, se diseñó un sistema de descarga en PDF mediante la codificación interna de imágenes en Base64 y generación de Blobs HTML, prescindiendo del uso de librerías externas y evitando bloqueos de seguridad del navegador por alertas automáticas de impresión.

5.2. Herramientas usadas

El desarrollo computacional del simulador se apoyó en un conjunto de librerías del ecosistema científico y de desarrollo de Python:

- **SymPy:** Fundamental para realizar cálculo simbólico. Permite computar de forma exacta las derivadas del bienestar social y de los sectores productivos, evitando errores de precisión flotante.

- **NumPy:** Empleada en la discretización de vectores numéricos y operaciones de *broadcasting* para generar el rango de datos que alimentan los gráficos bidimensionales a alta velocidad.
- **Matplotlib:** Utilizada para la renderización y dibujo dinámico de las curvas de costo fiscal, volumen de exportación e ingresos gubernamentales.
- **IPywidgets:** Proporciona el sistema de enlace dinámico para que los deslizadores, cuadros de texto y teclados virtuales interactúen en tiempo real con las funciones del motor matemático.
- **IPython:** Utilizada de manera complementaria para la integración de componentes de visualización HTML, inyección de estilos CSS y manipulación interactiva de las salidas gráficas directamente en el entorno de ejecución (Jupyter/Colab).

5.3. Explicación de los algoritmos de optimización

Para garantizar estabilidad numérica y dinamismo visual, el algoritmo no resuelve las ecuaciones simbólicamente buscando raíces exactas (lo cual podría fallar en funciones no lineales complejas introducidas por el usuario). En su lugar, el sistema emplea **SymPy** para calcular la primera derivada de forma analítica y rigurosa ($\frac{dQ}{dt}$ y $\frac{dC}{ds}$).

Luego, emplea el método `lambdify` para transformar estas ecuaciones simbólicas en funciones vectorizadas altamente optimizadas de **NumPy**. Esto permite evaluar todo el dominio posible de aranceles y subsidios (de 0 a 200 %) instantáneamente, usando `np.argmax` para hallar el óptimo empírico y evaluar su posición relativa frente al límite presupuestal definido.

5.4. Explicación de módulos del código

A continuación se analizan los segmentos cruciales de la estructura lógica del software desarrollado, divididos entre los motores principales de cálculo/interfaz, y los módulos secundarios encargados de la interactividad y exportación:

5.4.1. Módulo 1: Importación de dependencias y configuración del entorno

El pilar del simulador es la carga de librerías especializadas. Se inicializa el ecosistema gráfico bidimensional, las herramientas de inyección web (HTML/JS) para el entorno interactivo y el motor algebraico puro.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import ipywidgets as widgets
4 from IPython.display import display, clear_output, HTML, Javascript
5 import base64
6 import io
7 import sympy as sp
8
9 # Inicialización global de gráficos
10 # matplotlib inline
```

Listing 1: Inicialización del entorno computacional y dependencias

5.4.2. Módulo 2: Declaración de variables simbólicas y funciones base

Antes de procesar la interfaz, el núcleo matemático requiere declarar qué caracteres actuarán como símbolos algebraicos (variables independientes) para que el motor reconozca las derivadas. Asimismo, se establecen las estructuras canónicas predeterminadas de los sectores económicos.

```
1 # Declaración formal de variables simbólicas para derivación exacta
2 t_sym, F_sym, s_sym = sp.symbols('t F s')
3
4 # Funciones canónicas por defecto (modificables en la interfaz)
5 func_agro_base = "2000 - 0.5 * F * t^2"
6 func_tech_base = "3000 - 4 * F * t"
7 func_costo_base = "0.5 * s^2"
```

Listing 2: Definición del espacio algebraico continuo

5.4.3. Módulo 3: Construcción de la interfaz gráfica interactiva o GUI

Se instancia un sistema de deslizadores continuos (*Sliders*) y cuadros de texto (*Inputs*) mediante la librería *ipywidgets*. Estos controles son los que permiten al usuario interactuar dinámicamente con las variables macroeconómicas del país evaluado.

```
1 arancel_ext_slider = widgets.FloatSlider(value=25, min=0, max=200, step
    =1, description='Arancel Ext. (t) %:')
2 retaliacion_col_slider = widgets.FloatSlider(value=0, min=0, max=200,
    step=1, description='Retaliación (R) %:')
3 presupuesto_slider = widgets.FloatSlider(value=5000, min=0, max=20000,
    step=500, description='Presupuesto:')
4
5 # Elementos inyectados con clases CSS para formateo avanzado
6 casos_dropdown = widgets.Dropdown(
7     options=['Personalizado (Modo Libre)', 'Guerra comercial total (
    Efecto Búmeran)'],
8     value='Personalizado (Modo Libre)'
9 )
```

Listing 3: Instanciación de controles deslizantes macroeconómicos

5.4.4. Módulo 4: El algoritmo de cálculo híbrido Simbólico-Vectorizado

Este es el cerebro matemático del simulador. En lugar de depender de métodos iterativos lentos, el algoritmo emplea *SymPy* para obtener la derivada analítica pura y luego inyecta esa ecuación en la memoria plana de *NumPy* (*lambdify*) para evaluar instantáneamente todo el dominio fiscal.

```
1 # 1. Derivación Simbólica analítica (simulación continua)
2 der_agro_sym = sp.diff(expr_agro, t_sym)
3 der_costo_sym = sp.diff(expr_costo, s_sym)
4
5 # 2. Conversión C-based a funciones vectorizadas de ultra-alto
    rendimiento
6 f_agro = sp.lambdify((t_sym, F_sym), expr_agro, modules=['numpy'])
7 f_costo = sp.lambdify(s_sym, expr_costo, modules=['numpy'])
8
```

```

9 # 3. Evaluación matricial sobre el rango de subsidios posibles (0 a
    200%)
10 subsidios_rango = np.linspace(0, 200, 200)
11 Costo_Fiscal_Rango = f_costo(subsidios_rango)
12 Bienestar_Escudo = (Precio * f_agro(t_eff, F_val)) - Costo_Fiscal_Rango

```

Listing 4: Motor de optimización y derivación híbrida

5.4.5. Módulo 5: Renderizado del diagnóstico macroeconómico

El motor Matplotlib recibe los vectores precalculados y renderiza un lienzo bidimensional. Se implementa lógica condicional explícita (`fill_between`) para colorear en rojo las zonas donde el gasto público excede la recaudación (restricción fiscal).

```

1 fig, ax1 = plt.subplots(figsize=(10, 5))
2
3 # Dibujo de la curva de Bienestar Nacional
4 ax1.plot(subsidios_rango, Bienestar_Escudo, color='#2C3E50', linewidth
    =2.5)
5
6 # Trazado de la asíntota del óptimo matemático
7 ax1.axvline(x=subsidio_optimo, color='#27AE60', linestyle='--', label='Ó
    ptimo Matemático')
8
9 # Mapeo de la restricción del presupuesto (Sombreado rojo)
10 ax1.fill_between(subsidios_rango, 0, Costo_Fiscal_Rango,
    where=(Costo_Fiscal_Rango > presupuesto_max),
11               color='#E74C3C', alpha=0.15)
12
13
14 plt.show()

```

Listing 5: Renderizado gráfico de la frontera de posibilidades

5.4.6. Módulo 6: Gestor de eventos asíncronos y calibración en tiempo real

Para garantizar que cualquier cambio en la interfaz (mover un control) redibuje las curvas instantáneamente, se utiliza el patrón de diseño *Observer*. Se programan funciones de *callback* que se vinculan a los atributos `value` de cada componente.

```

1 def evento_actualizar(change):
2     # Desvincular temporalmente escenarios fijos si el usuario edita a
    mano
3     if casos_dropdown.value != 'Personalizado (Modo Libre)':
4         casos_dropdown.unobserve(evento_casos, names='value')
5         casos_dropdown.value = 'Personalizado (Modo Libre)'
6         casos_dropdown.observe(evento_casos, names='value')
7
8     # Renderizar todo nuevamente
9     actualizar_simulador()
10
11 # Asignar la subrutina a cada widget interactivo
12 for w in [arancel_ext_slider, retaliacion_col_slider, presupuesto_slider
    ]:
13     w.observe(evento_actualizar, names='value')

```

Listing 6: Enrutamiento de eventos mediante el patrón Observer

5.4.7. Módulo 7: Controlador paramétrico de escenarios económicos

Subrutina algorítmica encargada de modificar en cadena todos los deslizadores cuando el usuario escoge un escenario predeterminado en la interfaz, calibrando las variables según la teoría económica esperada.

```

1 def evento_casos(change):
2     caso = change.new
3     if caso == 'Libre mercado (Óptimo de Pareto)':
4         arancel_ext_slider.value = 0
5         retaliacion_col_slider.value = 0
6     elif caso == 'Guerra comercial total (Efecto Búmeran)':
7         arancel_ext_slider.value = 150
8         retaliacion_col_slider.value = 80
9
10    actualizar_simulador() # Llama al motor unificado

```

Listing 7: Calibración paramétrica predefinida

5.4.8. Módulo 8: Motor de compresión y exportación documental

Dado que los entornos web suelen bloquear funciones nativas por seguridad, este módulo extrae las figuras de Python, las convierte en una matriz de bits (BytesIO), las codifica en una cadena de texto Base64 pura, y luego inyecta directamente el código en el *Document Object Model* (DOM) mediante JavaScript para forzar el diálogo de impresión PDF.

```

1 def exportar_pdf(b):
2     buf = io.BytesIO()
3     # Guardar gráfico actual en buffer volátil
4     fig.savefig(buf, format='png', bbox_inches='tight')
5     buf.seek(0)
6
7     # Codificar la imagen para evadir restricciones de transferencia
8     img_base64 = base64.b64encode(buf.read()).decode('utf-8')
9
10    # Construir HTML fantasma e invocar API de impresión nativa del
    navegador
11    html_content = f"<html><body><img src='data:image/png;base64,{
    img_base64}'></body></html>"
12    display(Javascript("var w = window.open(); w.document.write('\" +
    html_content + "\"'); w.print();"))

```

Listing 8: Exportación a PDF mediante Blobs de JavaScript

5.5. Modos de simulación y escenarios

La inclusión de escenarios predeterminados responde a un propósito pedagógico. Dado que manipular múltiples variables dinámicas puede ser complejo para usuarios sin formación económica, estos “casos de estudio” guiados calibran el sistema automáticamente. Así, permiten observar de forma controlada el impacto macroeconómico de distintas decisiones políticas y aislar variables clave sin requerir ajustes manuales previos.

Los siete escenarios integrados en el simulador se explican a continuación:

1. **Personalizado (modo libre):** Otorga control absoluto sobre las variables matemáticas, sensibilidades y presupuestos. Es el entorno ideal para la experimentación abierta, permitiendo a los usuarios modelar hipótesis propias y evaluar condiciones de mercado específicas sin restricciones del sistema.
2. **Libre mercado (óptimo de Pareto):** Configura aranceles, retaliaciones y subsidios en cero. Actúa como línea base (*benchmark*) para demostrar empíricamente que, ante la ausencia de fricciones logísticas y costos burocráticos, la cooperación internacional maximiza el bienestar social de forma natural.
3. **Guerra comercial total (efecto búmeran):** Simula un fuerte ataque extranjero respondido con máxima retaliación estatal. Evidencia cómo el contraataque encarece agresivamente la logística interna (elevando el Factor de Guerra F), acelerando el colapso exportador y generando un daño autoinfligido que supera los beneficios de la represalia.
4. **Crisis estructural (colapso agrícola):** Combina un choque arancelario severo con una restricción drástica de liquidez en las arcas públicas. Ilustra matemáticamente cómo, al agotarse rápido el presupuesto, el Estado no logra alcanzar el óptimo de intervención, dejando a los sectores vulnerables ante una pérdida masiva de competitividad.
5. **Resiliencia tecnológica (shock asimétrico):** Contrasta directamente la vulnerabilidad del sector agropecuario con la estabilidad tecnológica bajo un mismo ataque. Explica visualmente cómo las industrias de alto valor agregado (demanda inelástica) amortiguan caídas comerciales sin requerir rescates fiscales masivos.
6. **Proteccionismo unilateral (absorción del golpe):** Analiza una estrategia política pasiva donde el país recibe un ataque pero mantiene su retaliación en cero. Al evitar el "efecto búmeran", se eliminan fricciones logísticas cruzadas, permitiendo que el presupuesto rinda más al enfocarse exclusivamente en subsidiar la industria local.
7. **Escudo fiscal perfecto:** Plantea un escenario teóricamente ideal con presupuesto abundante y alta eficiencia burocrática (bajo costo de inyección). Demuestra la capacidad analítica del Estado para neutralizar el daño externo, logrando mantener intacto el volumen exportador a expensas de un alto gasto público.

5.6. Resultados analíticos del modelo

La evaluación del sistema arrojó conclusiones macroeconómicas y operativas de carácter general, consolidando la utilidad de la modelación continua para la toma de decisiones:

- **Límites de la eficiencia fiscal:** Se comprobó que la viabilidad del gasto público decrece rápidamente ante fricciones externas prolongadas. Debido al comportamiento no lineal de los costos de intervención, existe un umbral general donde el costo marginal del subsidio supera el beneficio marginal de las exportaciones rescatadas, determinando el límite real de la capacidad de rescate del Estado.

- **Efectos de la fricción recíproca:** Los resultados demuestran que las políticas de retaliación comercial actúan como un multiplicador de distorsiones aduaneras y logísticas. Ante una agresión comercial bilateral, las pérdidas de bienestar nacional se aceleran debido al impacto acumulativo del factor de fricción, afectando con mayor severidad a los sectores de producción con mayor elasticidad.
- **Estabilidad del procesamiento digital:** A nivel computacional, el acoplamiento del motor simbólico analítico con la evaluación vectorial demostró una alta estabilidad matemática. El sistema resuelve de manera instantánea ecuaciones de optimización y restricciones de presupuesto sin experimentar sobrecarga de procesamiento, validando la eficiencia de la arquitectura implementada.

6. Conclusiones

El desarrollo del *Simulador Interactivo de Conflicto Arancelario* y el análisis matemático estructurado en este reporte confirman el cumplimiento integral de los objetivos planteados. Se logró construir una herramienta computacional robusta que traduce la complejidad de las guerras comerciales a un entorno visual e interactivo. Esto responde de manera directa a la problemática expuesta en la introducción: la necesidad de democratizar y facilitar el análisis macroeconómico avanzado para fines pedagógicos, permitiendo a cualquier usuario evaluar el impacto de las políticas fiscales sin requerir conocimientos técnicos previos.

Desde una perspectiva analítica, la implementación algorítmica del cálculo diferencial demostró empíricamente los límites de la intervención estatal. El modelo evidencia que los rescates fiscales ilimitados son insostenibles, y advierte que las medidas de retaliación (efecto búmeran) suelen agravar el daño logístico y productivo, especialmente en sectores de alta elasticidad. Finalmente, a nivel técnico, la hibridación de motores de cálculo simbólico y vectorial garantizó un procesamiento en tiempo real sumamente estable, validando la eficiencia de la arquitectura de software y dejando una base sólida para futuras expansiones del modelo.

7. Ficha técnica del software

A continuación, se detalla la configuración estructural y técnica del sistema computacional desarrollado. La arquitectura ha sido diseñada bajo un enfoque modular, priorizando el rendimiento del cálculo simbólico y la accesibilidad de la interfaz interactiva para el usuario final.

ESPECIFICACIONES TÉCNICAS DEL SISTEMA	
I. Información General e Implementación	
Nombre del Sistema	Sistema Computacional de Simulación Interactiva para Conflictos Arancelarios y Optimización de Política Fiscal.
Entorno de Ejecución	Entornos de computación interactiva basados en la web (Jupyter Notebooks, Google Colaboratory).
Lenguaje Base	Python (Versión 3.x o superior).
Licenciamiento Sugerido	Licencia de Código Abierto (MIT) orientada a la divulgación pedagógica y la investigación académica.
Repositorio de Código	https://github.com/emguerreror/proyecto-simulador-unal
II. Arquitectura Lógica y Motores de Procesamiento	
Paradigma de Interfaz	Interfaz Gráfica de Usuario (GUI) dinámica, generada mediante componentes <code>ipywidgets</code> para la manipulación de variables macroeconómicas en tiempo real.
Motor de Cálculo Simbólico	Resolución de ecuaciones, derivación analítica exacta y optimización matemática automatizada impulsada por la librería <code>SymPy</code> .
Motor de Cómputo y Renderizado	Procesamiento algebraico y matricial de alto rendimiento (<code>NumPy</code>) acoplado a la renderización gráfica vectorial bidimensional (<code>Matplotlib</code>).
III. Formatos de Salida y Presentación de Datos	
Módulo de Exportación	Generación de reportes estructurados con inyección HTML/CSS, representación formal de ecuaciones mediante LaTeX y exportación nativa a formato PDF mediante el aislamiento de ventanas de impresión (Blobs interactivos y codificación <code>Base64</code>).

Tabla 1: Resumen de especificaciones técnicas, arquitectura, herramientas y acceso al repositorio del simulador.

Referencias

- [1] Krugman, P. (1991). Increasing Returns and Economic Geography. *Journal of Political Economy*, 99(3), 483-499. Ver pág. 490, nota al pie de página 1, sobre la introducción de un colapso abrupto (“cliff”) en el sector agrícola bajo costos de transporte y fricciones comerciales.
- [2] Melitz, M. J. (2003). The Impact of Trade on Intra-Industry Reallocations and Aggregate Industry Productivity. *Econometrica*, 71(6), 1695-1725. Ver pág. 1700, Sección 3, sobre la estructuración de demandas agregadas bajo diferenciación de producto.
- [3] Samuelson, P. A. (1954). The Transfer Problem and Transport Costs, II: Analysis of Effects of Trade Impediments. *Economic Journal*, 64(254), 264-289. Ver pág. 268 sobre el modelo de “Costos de Iceberg” y fricciones proporcionales generadas por aranceles.
- [4] Harberger, A. C. (1964). The Measurement of Waste. *The American Economic Review*, 54(3), 58-76. Ver pág. 59, Ecuaciones 1 y 2, para la justificación del costo marginal cuadrático de intervenciones fiscales.
- [5] Brander, J. A., & Spencer, B. J. (1985). Export Subsidies and International Market Share Rivalry. *Journal of International Economics*, 18(1-2), 83-100. Ver pág. 88, Ecuación 7, sobre el uso de la derivada de la función de bienestar igualada a cero para encontrar el subsidio estatal óptimo.
- [6] Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace. Manual de referencia oficial del lenguaje de programación empleado como base del simulador.
- [7] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. Justificación del motor matemático de discretización y cómputo de matrices a alta velocidad.
- [8] Meurer, A., Smith, C. P., Paprocki, M., et al. (2017). SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3, e103. Referencia del motor simbólico encargado de realizar el cálculo analítico exacto de las derivadas del modelo.
- [9] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. Librería gráfica utilizada para la renderización de las curvas dinámicas de bienestar, costo fiscal e ingresos en el reporte exportable.
- [10] Grout, J., et al. (2015). ipywidgets: Interactive HTML widgets for Jupyter notebooks. *GitHub repository*. Recuperado de <https://github.com/jupyter-widgets/ipywidgets>. Cita referencial para la arquitectura de la interfaz gráfica y herramientas de control de usuario.
- [11] Pérez, F., & Granger, B. E. (2007). IPython: a system for interactive scientific computing. *Computing in Science & Engineering*, 9(3), 21-29. Justificación de las herramientas de maquetado interactivo, visualización HTML e inyección CSS en entornos Jupyter.